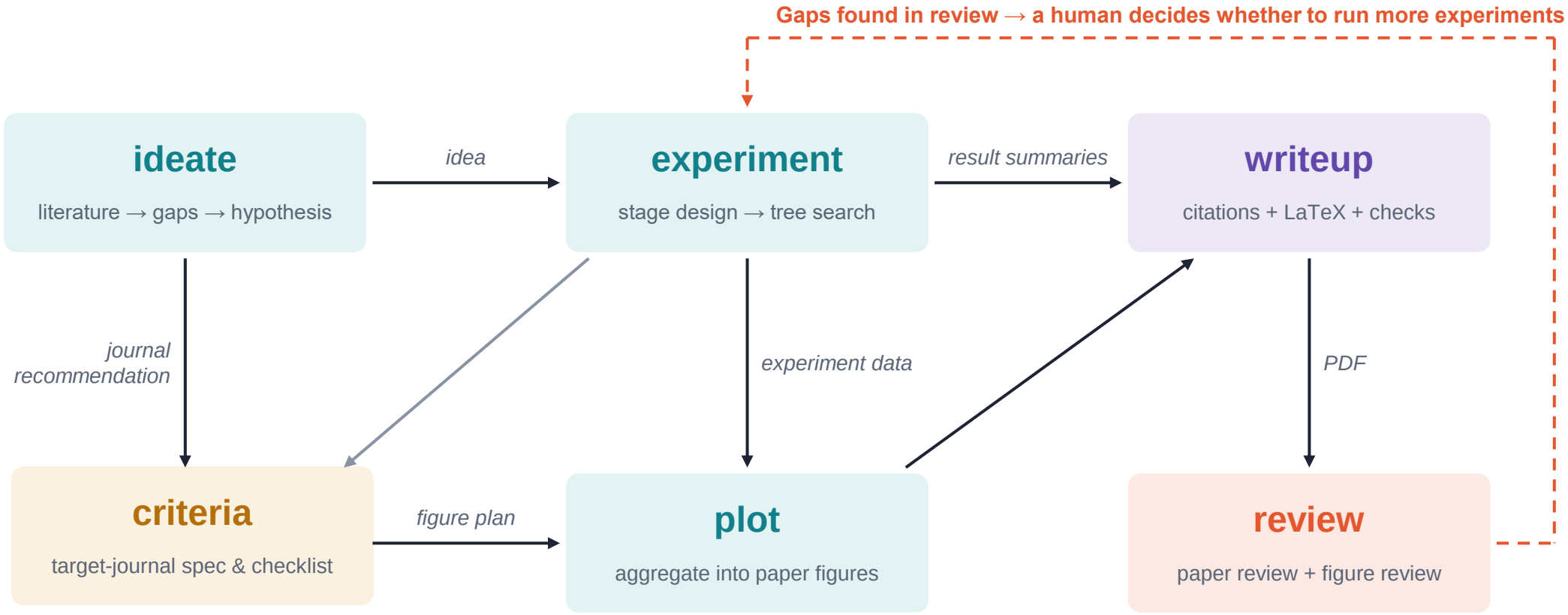
Our current research framework: six skills in one pipeline



Shared base One run directory per topic; every stage reads and writes there, and /ai-scientist chains the stages and resumes runs. Besides the figure plan for plot, criteria also sets the structure and length of writeup and the journal standard used by review.

ai-scientist-ideate

Stage A · 文獻基礎 —— 每個 topic 只跑一次 · 輸出檔已存在的步驟會跳過，可續跑

2 stages = A × topic · B × idea

Step	做什麼	Reference	Scripts	Output	失敗模式 / 陷阱
A1 frame	topic.md → PICO、3–5 個關鍵字的查詢（每個 outcome 一條）、錨點詞、來源、目標期刊、本機環境	scholar-pipeline.md	–	step0_session_config.json	查詢超過 5 個關鍵字：多數後端會把每個詞都 AND 起來，結果是 0 筆
A2 search	多資料庫檢索；跑完要讀每個來源的命中表與前幾筆標題	scholar-pipeline.md	`mcp_bridge.py search`	step2_raw_papers.json	量大不等於好。離題、某條 0 筆、漏掉錨點論文 → 用 `search --only Qn` 重寫那一條
A3 screen	三軸評分；composite ≥ 3.0 的全部保留	scholar-pipeline.md	`list-papers` `screen-build`	screening_scores.json step3_shortlist.json	自己算分數與排序：算術交給腳本，力氣放在評分與理由
A4 export	指派 citation_key，產生 BibTeX	scholar-pipeline.md	`ideate_tools.py export`	step4_references.bib step4_citation_keys.md	順序做錯：export 要在 screen 之後、snowball 之前。key 是之後每個 PDF 與全文的檔名
A5 snowball	找被最佳論文共同引用的經典；新增的論文再跑一次 A3 與 A4	scholar-pipeline.md	`mcp_bridge.py snowball`	step2b_snowball_papers.json	新增論文同樣要 relevance ≥ 3，否則人人都引的工具箱論文會混進來

A6 fulltext	挑 Tier 1 → 下載 → arXiv 補抓 → 補足名額 → 轉成 markdown → 擷取圖頁	scholar-pipeline.md	`tier1 · download`
			`arxiv-pdfs · pdf2md.sh`
			`figures · fulltext-status`
				pdfs/{key}.pdf
				step5_full_text/{key}.md
					抓不到 PDF 就記為 abstract-only 繼續做，不要停。API 金鑰不可印出或寫進受版控的檔案

A7 knowledge	主題與閱讀筆記 → 方法矩陣 → 點、面 → 線 → 描述性的體 → 概念索引	knowledge-passes.md
		knowledge-templates.md
			`build-methods`
			`build-cards`
			`concept-index`
				knowledge/ points · lines · planes · body
				methods_matrix.json
				concept_index.json
					卡片從來源填，不憑記憶；填不出就寫 (TBD)。筆記新增三行：**Claim scope / Assumes / Disagrees with**

三種深度：quick 3 條查詢、不讀全文（沒有 Lines，gap 證據弱，交付時要說明）　standard 5 條、前 20 篇全文　deep 7 條、全部 Tier 1

ai-scientist-ideate

Stage A 前半 · 文獻搜尋 —— 定義 → 搜尋 → 篩選 → 增補（回到篩選）→ 匯出 · 一個 Unit 一個決定

13 Units · L4 交接不過就拒跑

1 定義
A1 frame　協議先於結果

A1.p 協議
PRISMA-ScR · 依問題語意選框架 · 搜尋軸 = OR-cluster · 準則在搜尋前寫定
`frame set-protocol`

A1.q 查詢策略
查詢 = 軸的組合 · Boolean 與 flat 兩種寫法 · anchor · known items · scope probe
`set-strategy · probe`

A1.e 協議評估
獨立子代理讀協議；強制修一次 · 之後 accepted-with-doubts
`eval-brief · set-evaluation`

A1.v 情境
目標期刊、專家領域、本機資料與環境
`frame set-context`

寫出
`step0_session_config.json`
`step1_protocol.md`
`step1_protocol_evaluation.json`

frame check →

2 搜尋
A2 search　多源、可重播

A2.s 檢索
每個來源用它看得懂的寫法、依相關度取前 50 篇 · 每條查詢一個錄音實例 · dense D1
`mcp_bridge.py search`

A2.m 合併
確定性去重 · id 只追加 · 重跑過的查詢 · 舊實例的論文也保留 · SPECTER2 sem_sim
`mcp_bridge.py merge`

A2.c 逐條檢查
訊號由工具算；模型判 ok / replan / accept-weak · 模糊說法 keep / retire
`search-check · relevance-set`

寫出
`step2_queries/<id>@<hash>.json`
`step2_raw_papers.json`
`step2_relevance_check.json`

a2-check →

3 篩選
A3 screen　證據先於分數

A3.b 書目計量
標準化引用百分位 · 集合內被引 · SNIP → impact 用算的 · 不用判的
`mcp_bridge.py bibliometrics`

A3.g 機械門檻
X-RET 撤稿 · X-TYPE 非研究 · X-LANG（摘要是英文就放行）
`screen-gate`

A3 評分與名單
子代理看不到期刊與引用 · 先逐字引證據再打分 · 不合格的單筆退回重評
`screen-plan → merge → build`

寫出
`step3_bibliometrics.json`
`step3_gate.json`
`screening_scores.json`
`step3_shortlist.json`

a3-check →

4 增補
A5 snowball　補盲區

A5 向後 · 共被引
向後：≥ max(3, 2%) 篇納入論文引用 · 上限 40 · 共被引：只數引用 ≥ 2 個種子的論文 · 自成一池 · 上限 20
`mcp_bridge.py snowball`

A5 向前 · 語意相近
OpenAlex cites: 引用 ≥ 2 個種子 · S2 推薦 · 種子的判斷變了就重做該輪 · 成長 ≥ 10% 才有第二輪
`snowball [--round 2]`

實跑（2026-10-17 · 子代理評分）
搜尋 933 篇；S2 每條都被限流（必須揭露）
篩選 20 個子代理、1.83M token：1,050 篇納入 300
增補 117 篇；FASTER 經向後找回（52 篇引用）
模糊說法 automated cleaning：遠的全來自 Q3 → keep

寫出
`step2b_snowball_papers.json`
`_state/paper_ids.jsonl`（minted_by A5）

a5-check →

5 匯出
A4 export　只命名一次

A4.k Citation key
一篇一個 key · 終身不變 · 撞名取下一個字尾 · 退休不回收 · 用最完整的作者名單
`ideate_tools.py export`

A4.b BibTeX
Crossref 依 DOI 補齊類型、卷期、頁碼；查不到才用搜尋紀錄
`mcp_bridge.py bibliography`

出口契約 → 文獻融合（A6）
`a4-check` 通過才交出
· shortlist：納入只到標題 / 摘要層級
· 角色：奠基 · 核心方法 · 前沿 · 綜述
· 弱點清單 `search_caveats.json`
· key 從此是 PDF、全文與引用的檔名

寫出
`step4_citation_keys.jsonl`
`step4_references.bib`
`step4_bib_metadata.json`

重擬查詢、增刪說法（≤2 輪）

新增論文再篩 `relevance ≥ 3`

準則修訂 `screen-amend`：另記，不改協議

腳本　模型 / 子代理判斷　交接檢查：前一段沒過就拒跑

來源 PubMed · Scopus · IEEE · WoS · OpenAlex · arXiv · Semantic Scholar · Crossref　**本機** SPECTER2（CPU）

帳本 `_state/ledger.jsonl`：每個 Unit 記輸入 / 輸出雜湊，變了就往下游標過期 · `next` · `status` · `trace` · `preflight` · 人只透過 `_human/` 介入 · 網路回應可離線重播 · 13 個 Unit 都是 L4（12/12 · units check 機器判定）

交給文獻融合： `step3_shortlist.json` · `step4_references.bib` · `step4_citation_keys.jsonl` · `search_caveats.json` → A6 fulltext　|　舊主題（無 protocol）照舊流程

ai-scientist-ideate

Stage A 後半 · 文獻融合 —— 解構 → 對齊 → 碰撞 → 重構 · 每個 topic 一次 · 每句話指到一筆紀錄

15 Units · L4 交出研究問題清單

1 解構

A6 · A7.D　每篇只看自己

A6 全文

下載 PDF、arXiv 補抓、轉 markdown；表格數字看頁面圖 · 唯一連網的一步

`download · pdf2md.sh · figures`

A7.D 逐篇拆解

只讀自己那一篇：主張（強度、範圍）· 前提 · 限制 · 未來工作 · 沒做的事 · 實驗列 · 用語；引文逐字驗證

`decon-plan → merge`

實跑（golf50 驗收，四步合計）

解構 10 全文 + 40 摘要 → 179 主張、48 前提、66 實驗列
對齊 370 概念、227 座標、22 概念點
碰撞 138 候選全判 → 227 筆關係（conflict 13、前提類 58）
重構 18 缺口、13 條線、6 個面 → 5 題 ready
成本 ≈ 2.64M 子代理 token、82 分鐘
交接 Q_001 → gen_1 → consume → 下一題 Q_002

寫出
`step5_full_text/<key>.md`
`knowledge/decon/<key>.json`
`_notes/<key>.md`（視圖）

`decon-check`

2 對齊

A7.A · B1　先對齊，才比較

A7.A1 詞彙

一個概念一件事 · 同字異義拆成兩個、逐篇覆寫 · 兩篇定義的量測要寫等價說明

`align-terms → align-set`

A7.A2 座標

每個主張與前提六個座標：構念 · 量測 · 族群 · 資料 · 切分 · 方法類；工具建議 · 模型放

`align-coords --auto|--apply`

A7.A3 概念點

≥ 2 篇的用語或座標碰到的概念才成 Point —— 用算的，不用記的

`align-points → build-cards`

B1 視角

我們站在哪：資料在不在本機、方法類跑不跑得動、自己的資料；可行性只讀這份

`env-probe --write · lens-set`

寫出
`knowledge/alignment.json`
`methods_matrix_input.json`（投影）
`lens.json`

`align-check`

3 碰撞

A7.X · B3　每個比較都留紀錄

A7.X1 候選

工具依規則列出要比的組：同座標 · 共同前提 · 前提對主張 · 數字 · 演進 · 慣例對視角 · 空白格

`collide-candidates`

A7.X2 判定

每個候選恰一個判定：agree · conflict · boundary · extends · 前提類 … · 附兩邊差在哪些軸

`collide-plan → merge`

B3.c 視角碰撞

領域慣例 × 本機條件；空白格只算缺席 · 嚴重度 ≤ 3

`collide-plan --lens`

B3 缺口清單

每個缺口指到碰撞、≥ 2 篇支持；move 由主要碰撞決定；分數與排名用算的

`gaps-set`

寫出
`collision_candidates.json`
`collisions.jsonl`（只追加）
`step7_gaps.json`

`collide-check`

4 重構

A7.R · B2 · B3.5 · F.9　只組裝，不引入新事實

A7.R1 線 + 發現點

debate（≥ 2 方）· evolution（≥ 3 篇的鏈）· analogy；agree / boundary 升格成發現點

`recon-candidates`

A7.R2 面

主題在碰撞之後才分 · 每篇恰在一個面 · Q3 / Q5 引碰撞與線

`build-cards · build-methods`

A7.R3 描述性體

領域的七題；Q3 引演進線與 extends · Q5 引辯論線與衝突

`recon-check --unit A7.R3`

B2 綜整

topic 一份 SOTA：共識 · 爭議 · 報告數字都引碰撞或實驗列 id · 沒有就 (TBD)

`recon-check --unit B2`

B3.5 研究問題

依缺口排名：共同基礎 → 條件 → 後果 → 一個問題 → 篩檢；第 9 段交接攻擊目標

`questions-prefill · -handover`

F.9 索引與視圖

questions/index.json：ready 在前、依缺口排名 · fusion_map.md · concept-index

`questions-index · fusion-map`

出口契約 → 規劃塊（B4–B6，每個 idea 取一題）

`recon-check` 通過才交出
· ready 題數 ≥ max(代數, 3) + 2
· 弱點清單 fusion_caveats.json
· 第 9 段：攻擊目標 · 對手解釋 · 量級
`question-next → question-take → gen_k/step7b_problem.md`

寫出
`knowledge/lines/*.md`
`knowledge_cards.json`（點 · 面）
`knowledge/body/B_draft_*.md`
`step6_sota_review.md`
`questions/Q_NNN.md · index.json`
`knowledge/fusion_map.md`（視圖）

incommensurable → `align-set --reason`

B3 原文核對讀到更多 → A7.D 改紀錄 `decon-merge --revise` · 下游標過期

腳本　模型 / 子代理判斷　交接檢查：前一段沒過就拒跑

不變量 解構不跨篇 · 先對齊才比較 · 每個候選恰一個判定 · 重構每句引 id、不開全文 · 回頭只有宣告的修訂

帳本 `_state/ledger.jsonl`：寫檔的指令就是 Unit 的邊界，模型手寫的 6 個用 `unit-begin / unit-done` · 共用檔依部分擁有 · next · status · `trace X0001|GAP_001|Q_001` · 15 個 Unit 都是 L4（12/12）

交給規劃塊： `questions/index.json · step7_gaps.json · step6_sota_review.md · lens.json` → B4 假設　|　舊知識庫（沒有 knowledge/decon/）凍結，照舊流程

ai-scientist-ideate

Stage B · 論文規劃 —— Gap2Hypo → 設計 → 決策樹 → 攻擊型 Related Work → 封裝 · 每個 idea 一次

5 steps · 3 rounds 每步只讀前面的步

P1 Gap2Hypo
缺口決定假設的形式

C25 推導表
主要碰撞的結果定假設形式：conflict → 調節 · untested-shared → 前提檢驗 · extends → 增量；偏離要寫理由
`hypo-prefill`

C26–27 對手與數字
交接的對手解釋 → 帶 id 的對手（R1…）或寫明擱置；效應量進方向與 power
`hypo-check`

驗收（2026-10-18）
回溯 4/4 觸發 T09（判準 ≥ 3）
前進 golf Q_003 · EEG_SQ_v2 Q_001
盲審 4 次都偏好新版

hypo-check →

P2 實驗設計
能推翻 P1 的實驗

§4 設計
操作定義 · Scope · by-construction 六步 · 外部驗證；推翻條件要有資料做得到的邊界
`power · argument-render`

gate compute gate
在這裡先跑：本機跑得動、資料在不在；power.unit 跟著主張的範圍走
`env-probe · argument-check`

不變量
只往前讀；改上游就重產下游
C28 · T13 · R11 · K10 擋過期檔
7-key 契約不變 · 反思不連網

argument-check →

P3 決策樹
每種結果先寫好意義

T01–08 節點與分岔
每個決定性實驗一個節點；檢定必有「無法判讀」；正對照先於它授權的檢定
`tree-prefill`

leaf 葉與論文型別
finding · informative-null · boundary · rival-finding · descriptive · methods-caveat
`tree-render --project`

T09 決策閘
下限由 power 與正對照算出、只能調高；最可能的結果不能弱，弱結果 < 50%
`tree-check`

tree-check →

P4 攻擊型 RW
打誰、用什麼打

R01 攻擊目標
交接來的目標 = 某篇的某句話 + 它輸掉的碰撞；設計測不到的放 dropped
`attack-prefill`

R04–09 武器與落點
weapon E-id → lands_on 葉 · if_miss 另一片葉 · concede 讓步 · 措辭不強於原文
`attack-render → related_work.md`

B6 定稿
append-idea · consume --question
過不了閘：consume --abandoned
處方性 Body · 期刊（唯一連網）

attack-check →

P5 摘要標題封裝
最後才寫，只包裝

K01–05 摘要逐句
context → gap → approach → outcomes → contribution；每句標來源，≤ 250 字，不寫結果
`pack-prefill`

K06–11 標題與每葉封裝
標題只承諾已測部分；最可能的各種論文先備好標題與結果句 {{slot}}
`pack-render → idea_draft.json`

交給下游（另案 P-D16）
by_leaf 標題 → writeup
checkpoint → experiment gate_check
到達的葉 → criteria 第二輪

B5 反思
三回合包住五步

Round 1
P1 → P5 第一遍 + 四個檢查
凍結 *_r1（含樹、攻擊計畫）→ 盲讀者 1

Round 2
12 問 + 樹 4 問 → 一次修訂
`enters: P1 | P2 | P3` · 重做到 P3

Round 3
P4、P5 重做 → coherence gate
主張變了 → 盲讀者 2 → Finalize

寫出（gen_k/）
`step8_hypothesis.md`（P1 部分）
`argument.json`（主張 · derivation）
`p1_prefill.json`

寫出（gen_k/）
`step8_hypothesis.md`（其餘各節）
`argument.json`（設計欄位）
`argument.md`（視圖）

寫出（gen_k/）
`decision_tree.json` · `.md`
`claims[].outcomes`（樹的投影）
`decision_tree_r1.json`

寫出（gen_k/）
`attack_plan.json`
`related_work.md`（= Related Work）
樹的葉 → `targets_hit`

寫出（gen_k/）
`packaging.json`（`tree_gate`）
`idea_draft.json` → `idea.json`
`restatement_1/2.json`

腳本　模型 / 子代理判斷　檢查：沒過就不往下

論文型別 finding · informative-null · boundary · rival-finding · descriptive　**弱：** methods-caveat · not-publishable

規則 `references/planning/P1-P5.md`　**程式** `it_plan.py`　**測試** `tests/planning/`　卡片 P1–P5 在 L0（尚未接帳本）· 每代約 0.8–1.2M token、約 30 分鐘

從融合塊取題： `question-next` → `question-take Q_NNN` → step7b_problem.md + question.json（第 9 節：目標 · 對手 · 量級）

ai-scientist-ideate

Stage B · 研究循環（舊流程：沒有協議的主題）—— 每個 idea 跑一次 · 沒有人工檢查點 · 反思不碰網路

3 rounds 預設，最多 5 · 關卡 C1–C24

Step	做什麼	Reference	Scripts	Output	失敗模式 / 陷阱
B1–B2 persona · sota	逐字原版 persona 加專長與限制。先選一個沒用過的角度，再從那個角度綜述；設計與數字取自方法矩陣	research-cycle §1–2	`list-methods consumed`	step6_sota_review.md	先找 gap 才選角度：兩個 idea 會收斂到同一組實驗
B3 gaps	2–3 個有證據的 gap，各標一個 move（**延伸型 | 矛盾型**）；挑一組沒用過的 (gap, angle)	research-cycle §3 question-craft §3	`consumed`	step7_gap_analysis.md	「沒人做過」只是缺席，還不是值得問的問題。矛盾型要回全文確認來源真的那樣說
B3.5 problem	先查磁碟上有什麼。**共同認知 → 打破它的條件 → 後果**（So-what 階梯）→ 一個研究問題，過四道篩選	question-craft §2	`env-probe --write`	step7b_problem.md	後果只靠自己判斷時標 (TBD) 並記為風險，不能包裝成重要性
B4 hypothesis	RQ1 就是那個問題；**一個主要假設**；mode（test | estimate | equivalence）；預期方向、頭號競爭解釋、依構造的兩條式子、通往真實資料的路	research-cycle §4 question-craft §4, §6	`power` `argument-render`	step8_hypothesis.md argument.json → argument.md	反面明顯為假的主張是定義，不是假設。把弱點寫進風險不等於修好：否證要直接對著既有預測因子
B5 round 1	查詢（含 excluded）→ 7-key 草稿 → 檢查 → 凍結 _r1 副本 → **盲測轉述 1**（standard、deep）	research-cycle §5 ideation-reference §2.4	`argument-check --idea idea_draft.json`	idea_draft.json · argument_r1.json restatement_1.json	這一輪的 ERROR 留到第 2 輪一起修，不佔用關卡那一次強制修訂
B5 round 2	批判、比對轉述、**懷疑讀者 12 題** → ReviseHypothesis：每個 ERROR 與每個讀者疑慮都要有回應	question-craft §7	`argument-check --no-idea`	reflection_log.md tradeoffs · reader_doubts	刪掉實驗或主張卻沒有 tradeoffs 紀錄 → C19。用「只估計」避開檢查 → C21
B5 round 3	重寫草稿 → **compute**（加 **worth-it**）→ **coherence C1–C24** → 主要主張改過才做盲測轉述 2 → FinalizeIdea	question-craft §5	`argument-check --idea idea_draft.json`	idea.json（7 keys） restatement_2.json	每個關卡只強制修訂一次，之後照樣定稿並把疑慮寫進 Risk。縮小後答不了原問題：改寫問題，或放棄這組 (gap, angle)
B6 finalize	附加到 archive → 記入 (gap, angle) 帳本 → 處方性的體 → 期刊推薦（Stage B 唯一用網路的一步）→ 回到 B2	research-cycle §6–7	`append-idea` `consume · validate`	<topic>.json step8_journal_recommendations.md	Experiments 每條以編號開頭：`E3 [tests C1] …`。JSON 只放 7 個鍵，其餘留在 gen_k/

三道關卡：compute 跑得動嗎　worth-it 縮小後還答得了問題嗎　coherence argument-check C1–C24。另有盲測轉述，最多兩次。輪數上限 = num_reflections + 2

ai-scientist-criteria

目標期刊分析 · 跑兩輪：第一輪在實驗前、第二輪在實驗後 · 無人工檢查點

criteria.json schema /2 · pass 1 → 2

第一輪 PROCEDURE —— 實驗前（查過的期刊只做最後一步）

S0 — **選期刊** `pick-venue`
ideation 推薦的第一本，或 --venue；期刊層已存在就整層沿用

S1 — **核心理念 + 期刊檢核表（S4v）** `WebSearch / WebFetch`
每句標 [OFFICIAL] / [THIRD-PARTY] 與日期；§05 寫成期刊通用的「任何論文都要做到什麼」

S2 — **範例論文：只取自目標期刊** `venue-scan → 評分 A / B → select`
A 主題、B 問題形狀、C 章節安排、D 證據風格；同分時 B 優先

S3a — **全文與章節樹** `fetch-fulltext → measure → exemplar-gate → structure-check`
五條全文路線；讀不到就換下一名；每個子章節附一句逐字錨點證明讀過

S3b — **融合結構（子章節等級）** `step3b_fused.json → structure-render`
功能區塊 · 幾篇範例有 · 字數中位數 · 最佳範本 · 共同做法（shared_practices）

S4t — **期刊 LaTeX 模板** `fetch-template → adapter.tex → template-smoke`
用 tectonic 試編譯，量出欄寬與每頁字數，確認 AI 揭露印在第一頁

S3c — **大綱與交接（S4r）** `init-criteria → sync-gaps → outline-render → validate`
融合結構 × idea 的 Experiments 與假設；圖表位全是 planned；列出 readiness gap

第二輪 —— 實驗後，只重做 S3c + S4r `init-criteria --pass 2`
圖表位 planned → ready / needs-data；Results 標題定稿；gap 狀態從 experiment_plan.json 同步。
所有 stage 結束（或 gated-off）才能產生；plot 與 writeup 只接受 pass 2。

三層資料 —— 期刊的事實只查一次

期刊層 — `ai_scientist/venues/<slug>/` *所有 topic 共用*
venue.json · S1 · S4v · template/ · exemplars/（全文、章節樹、量測）

Topic 層 — `…/topics/<topic>/` *取決於「像不像我們的論文」*
候選與評分 · exemplar_gate.json · step3b_fused.json

Run 層 — `<run>/criteria/` *每個 run 重做*
criteria.json · final_outline.md · checklist.md · venue_pointer.json

criteria.json 交給誰

欄位	內容	誰讀
sections[].subsections[]	標題 · 主張 · 寫法重點 · 證據 · 圖表 · 字數 · 範例出處	writeup
figures / tables .slots	每張圖表給讀者看什麼、來自哪個 stage	experiment · plot
readiness_gaps[]	期刊期待但研究沒有的；needs 決定誰接手	experiment（規則 9）
template · limits	class、欄寬、字數上限、試編譯結果	plot · writeup

關卡 —— 每條規則都有工具檢查

① **exemplar-gate** 可讀範例不足就自動補下一名；不到 3 篇 → 降級並寫進報告
② **structure-check** 錨點要能在全文找到，標題覆蓋 ≥ 90%
③ **validate** Results 一節一個發現；每個假設判定與負結果各自一節；字數加總 ±10%
④ **provenance** 從別的 run 複製 criteria.json 直接報錯
⑤ **exp_plan.py validate** 硬規則 9：gap 沒有處置就不能開跑
⑥ **舊 pack** <topic>_research/venues/ 凍結，只讀不寫

產出： `<run>/criteria/criteria.json`（pass 1）→ 交給 /ai-scientist-experiment：每個 gap 要有處置 → 實驗後第二輪（pass 2）→ 交給 /ai-scientist-plot

ai-scientist-experiment

4 stages × (best-first tree search) · 每個 node 7 步 · 我扮演 manager + worker + reviewer + VLM

good = is_buggy=false AND is_buggy_plots=false

四個 STAGE（每個 stage 跑同一個迴圈）

Stage	目標	動作	iters 原版→你	完成條件
1 initial implementation	能跑起來的最小實作	draft ×3 → debug	20 → 4	出現 1 個 good 就結束 *(原版先判 max-iters)*
2 baseline tuning	調參 + 加資料集 不准改架構	hyperparam (從 root 分支)	12 → 3	max-iters，或 best≠root 且你判定穩定、≥2 資料集
3 creative research	真正的新東西	improve → debug	12 → 3	max-iters，或 best≠root；跑太快會被要求加重實驗
4 ablation studies	逐一拆掉元件看貢獻	ablation 一個 node 一項	18 → 4	只看 max-iters

stage N 的 best node → 複製成 stage N+1 的 root（seed-root，不算新嘗試）

選點策略 —— journal.py next（逐行移植，不要自己想）

① **draft** 若 root 數 < num_drafts(3) — 只有 stage 1，2/4 設 0
② **debug** 機率 debug_prob=0.5，挑 buggy 葉節點，depth ≤ 3
③ **improve** stage 1/3：改進最佳 good node
④ **branch** stage 2/4：一律從 seed-root 長出新的參數/消融想法

實際跑過的證據（FLUTE run）

stage 3 真的觸發了 debug 鏈：**improve → improve(IndexError) → debug → good** 29 個 node、26 個 experiment_data.npy、4 份 summary

WORKER PROCEDURE —— 一個 node 的 7 個步驟

1 Plan + code `journal.py add`
依 action 選 prompt：draft §4.2 / debug §4.3 / improve §4.4 / hyperparam §4.5 / ablation §4.6

2 Execute `run_node.py`
隔離工作區跑 code，吃 exec_timeout；輸出 experiment_data.npy + PNG

3 Analyse `journal.py set`
讀 output 判 is_bug；有 exception 一律 buggy

4 Metrics `npy_tree.py`
拆 .npy 取每個 dataset 的最終值；沒有有效 metric ⇒ buggy

5 Plots `run_node.py --what plot`
寫繪圖碼存進 working/（buggy 就跳過）

6 看圖 (VLM) `journal.py set`
用 Read 開每張 PNG，判 is_buggy_plots — 這是 good 的硬門檻

7 Export `journal.py export`
更新 tree_plot.html / best_solution

四個真正的陷阱

① **metric 平均值會騙人** — stage 2 多測資料集反而拉低平均。實測 4e7f190f 平均 1.93 卻輸給 1.81 的 b29af507：要逐資料集比，然後 pin-best
② **沒判過圖的 node 不是 good** — is_buggy_plots=None ≠ false（EEG run 真的有一個）
③ **max_iters 數的是全部 node**（含 seed-root） ④ **種子別寫死** — 要讀 AI_SCIENTIST_SEED，否則多種子評估是假的

產出： `logs/0-run/{draft,baseline,research,ablation}_summary.json` → 交給 /ai-scientist-plot。每個 stage 結束都會對 best node 跑 num_seeds 次多種子評估 + 一個聚合 node（mean±std）

ai-scientist-plot

資料圖（aggregator）+ 示意圖（schematic branch）· 共用一個狀態檔與一個 gate · 反思 ≤5 輪

每個 ready slot 一張 無 criteria 時 ≤12 張

資料圖 PROCEDURE —— 六步（第 5 步用 --only 回頭只重畫有問題的圖）

0 準備 `cp plot_style.py plot_panels.py → <run>/`

只清資料圖的 .png .pdf .svg .fig.json · 示意圖與 _work/ 保留；rsync 結果鏡像到 <run>/experiment_results/

新模組只給新的或要重畫的run · 不要放進已完成的run

1 先看資料長相 `npy_tree.py … --values`

讀 experiment_data.npy 的 key、shape、末值；有設計 stage 的 run 再讀 stage_summaries/index.json

猜 key 名是第一大失敗原因；一個假說一張圖或一組 panel

2 決定形式 `references/figure-forms.md`

每個 slot 先寫：主張、資料形狀、單位與數量、至少兩個候選與淘汰理由、選定的形式 → 狀態檔的 form

criteria 的 plot_type 只是提示；統計量來自多個單位時，預設畫出個別資料點

3 寫 aggregator `plot_style.run_figures(FIGS)`

一張圖一個回傳 Figure 的函式；panel 取自 plot_panels；開頭一張 STYLE 樣式表全篇共用

配色四種：角色 SEMANTIC · 有序 ramp(n) · 類別 categorical(n) · 超過 6 類改灰階加標記

4 執行與存檔 `finalize() → .fig.json`

PY auto_plot_aggregator.py 2>&1 | tee aggregator_output.txt；每張圖照期刊欄寬精確存成 PNG + PDF + SVG

紀錄檔由程式量出：實際寬度、印出後最小字級、用到的顏色、版面警告；--check-determinism 重畫比對

5 反思（≤5 輪） `--only F3,F5`

用 Read 看每張 PNG 與 .fig.json；不看圖說，幾秒內看得出主張嗎；只重畫有問題的圖

字級放不下就減panel 或拆圖（<id>b + split_of）· 不縮字；沒畫出的內容記在 dropped

GATE —— paper_bridge.py figure-gate（回報前必過；只檢查有紀錄檔的圖，舊 run 不受影響）
紅燈 寬度差 >1 mm · 超出畫布 · 印出字級低於期刊下限 · 色票外顏色 · 清單外 colormap · 重畫結果不同 · 版面塌陷
警告 上下標字級 · 圖高 >225 mm（補充圖約 188 mm）· 圖清單與 criteria 不符 · 放進稿件時被縮小 · 缺 form
示意圖 批評 ≥2 輪 · SVG ≥4 輪 · 稽核通過 · 有參照圖 · 向量 PDF 內嵌字型

示意圖 SCHEMATIC BRANCH —— plot_type: schematic 的 slot，六步

步驟	做什麼	工具 / 規則
brief	內容只取自實際跑過的程式：列出每個 stage 的程式與它載入的模組	`paper_bridge.py brief-facts`
參照	期刊範例的主圖 → 其他 topic 的同期刊範例 → 文獻庫 → 網路；只取慣例，不抄內容	`pdf_tools.py figures`
渲染	TS_FIG_ENGINE：PaperBanana → gen_image → gemini-web → Claude 自己畫 SVG	`gen_image.py · gemini_image.py`
批評	至少 2 輪，看著實際渲染逐面向找紅線；方法圖不放結果數字	`schematic-craft.md`
重畫	重畫成可編輯的 SVG，至少 4 輪稽核；字型跟期刊（Times 或 --font sans）	`audit_svg.py · render_svg.py`
出貨	向量 PDF + PNG；狀態檔記錄引擎、參照圖、輪數、slot_conflicts	`svg_to_pdf.py`

檔案歸屬 —— 全部在 ai-scientist-plot/ 底下，不再讀 vendor

scripts/	`plot_style.py · plot_panels.py · panel_gallery.py · audit_svg.py · render_svg.py · svg_to_pdf.py · gen_image.py · gemini_image.py …` *10 支來自 spark-to-paper（MIT，見 THIRD_PARTY.md）；10 種 panel 的範本庫是自寫的*
references/	`data-figure-style · figure-forms · panel-gallery · schematic-branch · schematic-craft · svg-craft` *規則改寫成自己的用語：資料圖三份，示意圖三份*
<run>/	`auto_plot_aggregator.py · plot_style.py · plot_panels.py · figures/ · figure_plan_status.json` *模組複製進 run：一支腳本加兩個檔案，搬到哪裡都能重跑*

四個實測學到的事

① **seed 0 會被算兩次** best node 和 seed-0 複本是同一次執行，mean±std 只能算一次
② **類別色在灰階下分不開** 最小亮度差只有 0.8；`categorical()` 給的標記與線型一定要用
③ **8 pt 下限會改變圖的密度** MI 實跑：12 張全是 150 mm 雙欄，最高的一張 218 mm
④ **長條圖 y 軸規則不變** 照舊縮到相關範圍；紀錄檔只記下座標軸有沒有包含 0，不做檢查

產出： `figures/F<n>_<name>.png · .pdf · .svg · .fig.json`（示意圖另有稽核與修復紀錄）+ `figure_plan_status.json` + `auto_plot_aggregator.py` → 交給 /ai-scientist-writeup

ai-scientist-writeup

journal 模式 J1–J10：準備 → 起草 → 精修 → 關卡 → 對抗式審查 → 定稿 · 另有 --revise 修訂入口

小數一律是 {{token}} 由 npy 算出再代入

JOURNAL 模式 —— J1 到 J10（有 criteria.json 時的預設）

J1–2 準備素材 `bib_tools.py · s2_search.py · doi2bib.py`

引用：ideation 的 bib 當種子，再用 Semantic Scholar 補（≤20 輪）；圖：逐張看過，寫下每個比較的方向

條目不憑記憶手寫；描述存成figure_descriptions.json，審稿人靠它「看」圖

J3 寫作前 `template-json · facts · venue-template`

字數帶（節 ±25%、小節 ±35%）· blueprint.json（貢獻、核心論點、章節角色）· facts_spec.json · 期刊 class

每個小數都要有來源：npy、推導式，或宣告的文獻值；class 用不了就退回通用版面並記下原因

J4 起草 `references/drafting-rules.md → build`

照 criteria.json 的小節逐一寫進 latex/src/；先寫 Methods 與 Results，摘要最後寫；圖表宣告在 figures.json

每種章節角色有不該寫的：Results 不寫因果與詮釋，Methods 不比較，Conclusion 不引入新資訊

J5 精修 `snapshot → refine.md → preserve-diff → lock`

朝字數帶擴寫（只用 section-context 列出的素材）· 去 AI 腔 · 語氣對應證據強度 · 追蹤核心論點

token、引用、ref、label、公式必須和快照完全相同；AI 揭露句不算AI 腔，不能動

J6–8 關卡與審查 `gates · review-text · cite-support`

gates 全綠 → 隔離的審稿人讀全文（issue 附逐字引句）→ 三個角度反駁 → 修正 → 引用對照全文 → 再跑 gates

需要新資料的意見只寫成readiness gap（RG-R），不會自動開實驗*

J9–10 定稿 `figure-gate · compile_latex.py`

示意圖與 Method 同步 → 反思 3 輪（編譯、看頁面、查頁數），每輪結尾跑 gates → <runname>_journal_final.pdf

最後一次gates 綠燈才出定稿；關不掉的紅燈照實回報，不硬湊

GATE —— paper_bridge.py gates（J6、J8 與每輪反思之後都要跑）
紅燈 數字不在 facts · AI 腔片語 · 字數帶 · 小節缺漏 · 引用缺條目 · 揭露不在第 1 頁 · src 改了沒 build · 論點被否定
advisory 永不變紅：摘要數字不在 Results · 貢獻沒出現 · 章節角色用語 · 強動詞配弱證據 · 引用不被原文支持
鎖 lock --step refine | review | final 記下 src 的雜湊；之後改過的檔案只補做該補的檢查

--revise 修訂入口 —— 審查與人的意見回到稿子（一次一輪，不自動再審）

步驟	做什麼	工具 / 規則
R1 匯入	期刊標準審查、圖的審查、review_input.md → 同一份 issues.json；NeurIPS 表單的審查要 --from final 才進來	`issues.py ingest`
R2 定位	補上章節、稿子裡的原句、一句關閉條件；要修的先定位再動手	`issues.py set --quote`
R3 分類	馬上修 · 留給作者（→ RG-R*）· 丟棄並寫理由；人的意見預設接受	`issues.py set --state`
R4 修改	只改指名章節、只做滿足關閉條件的最小修改 → build → gates → 登記落地的句子	`refine.md §6`
R5 收尾	沒有 open 的 issue；補做受影響章節的檢查；舊定稿改名 .v<k>.pdf 保留	`issues.py check · lock`

檔案歸屬 —— 全部在 ai-scientist-writeup/ 底下，.claude/vendor/ 已不存在

scripts/ `draft_lint.py · citations_lint.py · reflow_tex.py · doi2bib.py · issues.py · review_panel.workflow.js`
issues.py 以外的五支來自spark-to-paper（MIT，見 THIRD_PARTY.md）

references/ `drafting-rules · refine · adversarial-review · revise · workshop-mode · writeup/`（原版 prompt）
起草、精修、審查、修訂各一份；章節角色與修訂流程參考SCHOLAR 的journal 系列

<run>/latex/ `src/ · sections/ · results.facts.json · issues.json · logs/`
只編輯src/，sections/ 由build 產生；logs/ 有gates 報告、鎖、審查與引用檢查紀錄

五個實測學到的事（2026-10-05）

① **審查的驗證幾乎不淘汰** lean 檔實跑 71 筆 issue，沒有一筆被兩個角度反駁
② **引用只查得到四分之一** P300：61 篇裡 14 篇有全文，39 句中 10 句只被部分支持
③ **因果詞規則只有三成五是真的** 43 筆命中、15 筆成立，所以只當 advisory
④ **人的意見不會被丟棄** 做不到就標 `cannot_apply` 並寫原因；工具拒絕 dropped
⑤ **還沒有新 run 走完整個新流程** 目前的驗證都是在既有 run 的副本上做單一步驟

產出： `<runname>_journal_final.pdf · latex/src/ · results.facts.json · issues.json` → /ai-scientist-review ｜ 無 criteria.json：workshop 模式（4 或 8 頁）

ai-scientist-review

兩份獨立的審查：A 讀全文給分數　B 逐張看圖對照說明與內文

負面先驗是刻意的　壓力測試，不是場次判決

PART A —— 文字審查（NeurIPS 表格）

1 抽全文

pdf_tools.py text --pdf <paper> --out paper_text.txt

pymupdf4llm → pymupdf → pypdf，每層不足 100 字就往下

2 校準

讀 132_automated_relational.json 的 review 欄位

num_fs_examples=1；只學語氣與分數尺度，不抄內容

3 扮演審稿人

reviewer_system_prompt_neg：不確定就給低分、拒稿

AI 生成的論文傾向過度宣稱，所以先驗是負的

4 填表 + 輸出

THOUGHT: 段落 → REVIEW JSON: 區塊，16 個欄位固定順序

存成 review_text.txt（副檔名是 .txt 但內容是 JSON）

REVIEW JSON —— 16 個欄位，順序固定

Summary · Strengths[] · Weaknesses[] · **Originality / Quality / Clarity / Significance 1-4** · Questions[] · Limitations[] · Ethical Concerns bool · **Soundness / Presentation / Contribution 1-4** · **Overall 1-10** · **Confidence 1-5** · **Decision 只能 Accept / Reject**（不接受 Weak / Borderline）

有機制但預設不跑

num_reviews_ensemble=1（多審稿人平均 + meta-review）　**num_reflections=1**（自我修正輪）

launch_scientist_bfts.py 用全預設呼叫 → 一次審查、一個 few-shot、無反思、無集成。要用得自己開。

PART B —— 逐張圖審查（原版用 VLM，你用 Read）

`pdf_tools.py figures --pdf <paper> --out-dir <stem>_imgs --summary`

每個 figure 切出 {img_name, caption, images[], main_text_figrefs[], page}，然後用 Read 真的把 PNG 打開看，回答四個欄位：

`Img_description` 這張圖畫了什麼　`Img_review` 圖本身的問題（圖例、標籤、看不看得懂）

`Caption_review` 說明是否準確　`Figrefs_review` 內文有沒有把這張圖講清楚

Part B 抓到 Part A 看不到的東西

FLUTE figure 1 的 Caption_review：

「說明裡的數字來自單一 seed，但內文報的是三個 seed 的平均（0.529/0.490/0.549）」

這種跨檔案的不一致，只有把圖、說明、內文同時擺在一起才看得出來。

兩份論文的實際評分

	Orig/Qual/Clar/Sig	Sound/Pres/Contr	Overall	Conf	Decision	圖有意見
FLUTE 壓縮	2·3·3·2	3·3·2	4	4	Reject	6 / 8
EEG 品質	3·3·3·2	3·3·2	5	4	Reject	2 / 9

Reject 不代表這篇論文不好

原版對每一篇都套 NeurIPS 主會議表格 + 負面先驗，不管它其實投的是 4 頁 workshop。這是壓力測試。

FLUTE 的 review_thought.txt 自己講得很清楚：

「對 pitfalls workshop 而言這研究是合適的；對頂級 ML 會議而言貢獻太窄、資料是模擬的、新穎性低。帶著負面先驗我給拒絕。」

報告時要講清楚這一點，不然會誤判自己的論文。

產出：`review_text.txt · review_thought.txt · review_img_cap_ref.json · <stem>_imgs/*.png`　挑哪個 PDF：有 final 的優先 → 否則 reflection 編號最大 → 否則任一